MQTT API events - documented vs actual

`FXR_60-90_mqtt_api.json` documents the reader's asynchronous events - management events under `/async-events` and tag data under `/tagDataEvents`, `/mode_tag_data_events` and `/directionality_tag_data_events`. Both were compared against events captured live from a reader.

Tag data events are broadly accurate. Management events are not - the heartbeat example has drifted and six of seven event types have no example at all.

Document	FXR_60-90_mqtt_api.json - Zebra Fixed Reader MQTT API v1.0.0
Reader	10.233.48.36 - FXR60, reader application 5.0.7
Date	2026-09-07
Management events captured	13 - 12 x gpo, 1 x heartbeat
Tag data events captured	228 across 3 configurations
Raw captures	captured_events.ndjson - tagdata_plain.ndjson - tagdata_full_metadata.ndjson - tagdata_tid_user_xpc.ndjson

How the events were captured

The reader publishes management events to whatever is configured in `endpointConfig.management.event`. On this reader that pointed at AWS IoT, which cannot be read back, so a local MQTT connection was added alongside it.

`PUT /cloud/config` cannot do this - it accepts only `data`:

```
{"code":1,"message":"Invalid endpoint configuration not a vaild schema"}
```

`PUT /cloud/cloudConfig` was used instead, preserving the existing `control`, `data` and `management.commandResponse` blocks and adding:

```
{"endpointConfig": {"management": {"event": {"connections": [{
  "name": "MGMT_EV_LOCAL", "type": "mqtt",
  "options": {"enableSecurity": false,
              "endpoint": {"hostName": "10.117.229.18", "port": 1883, "protocol": "tcp"},
              "additional": {"cleanSession": true, "clientId": "fxr60-mgmt-ev",
                             "keepAlive": 60, "qos": 0},
              "publishTopic": ["fxr60-lab/mevents"], "subscribeTopic": []}}]}}}}
```

Then subscribed and triggered GPO transitions:

```
mosquitto_sub -h 10.117.229.18 -p 1883 -t 'fxr60-lab/mevents'
PUT /cloud/gpo   {"port": 1..4, "state": true}   then false
```

The management event configuration was already fully enabled on this reader - `gpiEvents: true`, `gpoEvents: true`, `heartbeat` with all fields, and every `errors` category.

1. Only one of seven event types has an example

The `/async-events` description documents seven `type` values and links each to its own payload page:

type	Documented	Example in the file

heartbeat	[OK]	[OK] example1
gpi	[OK]	[X] none
gpo	[OK]	[X] none
error	[OK]	[X] none
warning	[OK]	[X] none
firmwareUpdateProgress	[OK]	[X] none
userapp	[OK]	[X] none

The whole point of the envelope is that `data` changes shape per `type` - the document says so itself:

> **Note:** Always check the `type` field before parsing `data`. The `data` shape is different for each event type.

With one example, six of the seven shapes are undocumented. A `gpo` event is small enough to show in full:

```
{
  "component": "RG",
  "data": {"pin": 1, "state": "HIGH"},
  "eventNum": 9,
  "timestamp": "2026-09-07T12:21:53.635+0000",
  "type": "gpo"
}
```

That is the actual captured payload. Nothing in the document shows it.

---

2. The heartbeat example is out of date

Diffing the documented `example1` against a live heartbeat, ignoring per-antenna numeric keys:

Fields the reader sends that the example does not show

Field	Live value
data.system.GPI	{"1":"HIGH","2":"HIGH","3":"HIGH","4":"HIGH"}
data.system.GPO	{"1":"HIGH","2":"LOW","3":"LOW","4":"LOW"}
data.system.hostName	present
data.system.macAddress	present
data.system.powerSource	e.g. PWR_BRICK
data.system.powerNegotiation	e.g. POE+
data.reader_gateway.interfaceConnectionStatus	control / data / management / managementEvent arrays, each with per-interface stats
data.reader_gateway.dataPathStatistics	array of per-path counters
data.reader_gateway.numDataMessagesRxedFromExt	present

`interfaceConnectionStatus` inside the heartbeat is the most substantial omission - it reports the live connection state of every configured endpoint, with command/response counters:

```
"interfaceConnectionStatus": {
  "control": [{"connectionError": "", "connectionStatus": "connected",
               "description": "Control cmd/rsp through AWS IoT Core",
               "interface": "CTRL_AWS",
               "stats": {"commands_received": 0, "responses_failed": 0,
                         "responses_sent": 0}}],
  "data":    [{ ... }],
  "management": [{ ... }],
  "managementEvent": [{ ... }]
}
```

That is exactly what a monitoring backend needs from a heartbeat, and it is absent from the documentation.

Fields the example shows that the reader does not send

Field	Status
data.reader_gateway.numDataMessagesRxed	replaced by dataPathStatistics[]
data.reader_gateway.numDataMessagesTxed	replaced by dataPathStatistics[]
data.reader_gateway.numDataMessagesDropped	replaced by dataPathStatistics[]
data.reader_gateway.numDataMessagesRetained	replaced by dataPathStatistics[]
data.system.flash.	flash is present but null on this reader

The four flat `numDataMessages*` counters have moved into an array:

```
"dataPathStatistics": [
  {"noLockQDepth": 0, "numDataMessagesDropped": 0, "numDataMessagesRetained": 0,
   "numDataMessagesRxed": 0, "numDataMessagesTxed": 0}
]
```

A client written to the documented shape reads `numDataMessagesRxed` at `reader_gateway` level and finds nothing there.

A casing mismatch

Document	data.system.systemTime
Reader	data.system.systemtime - lowercase t

Live value: `"07/09/2026 12:22"`. A consumer keying on `systemTime` gets nothing. Note this is also **not** the ISO-8601 format used by the envelope's own `timestamp` field (`2026-09-07T12:21:53.635+0000`) - two different time formats in one event.

Antenna count

The example shows **13** antennas (`"1"` ... `"13"`). This FXR60 reports **6**. Not an error - the count is device-dependent - but an example with 13 entries suggests a fixed shape where the real one varies by model.

---

3. flash is null

The example populates `data.system.flash` with four sub-objects (`platform`, `readerConfig`, `readerData`, `rootFileSystem`), each with `free` / `total` / `used`. The reader sends:

```
"flash": null
```

The same is true of `GET /cloud/status`, where every flash figure reads zero. So either the reader does not populate it

on this hardware or firmware, or the field is deprecated. Either way the example does not match.

---

4. Tag data events - largely accurate

Tag data was captured in three configurations and compared against `/tagDataEvents example1`.

The envelope matches

	Doc	Live
Top-level keys	type, timestamp, data	type, timestamp, data

11 of the 14 documented `data` fields appeared live exactly as named:

```
CRC  PC  accessResults  antenna  channel  eventNum
format  idHex  peakRssi  phase  reads
```

A real event, plain `CUSTOM` mode with no `tagMetaData`:

```
{
  "data": {"eventNum": 4808, "format": "epc",
           "idHex": "e2806894000040017790e471"},
  "timestamp": "2026-09-07T12:29:55.456+0000",
  "type": "CUSTOM"
}
```

And with metadata plus a TID access operation:

```
{
  "data": {"CRC": "f3b3", "PC": "3000",
           "accessResults": ["e280689420004001"],
           "antenna": 1, "channel": 908.25, "eventNum": 4828,
           "format": "epc", "idHex": "e2806894000040017790e471",
           "peakRssi": -37, "phase": 14.029969215393066, "reads": 1},
  "timestamp": "2026-09-07T12:30:28.390+0000",
  "type": "CUSTOM"
}
```

type reflects the mode, not a fixed value

The documented example shows `"type": "SIMPLE"`. Live it was **`"CUSTOM"`** - matching the `type` set in `PUT /cloud/mode`.

So `type` is the operating mode (`SIMPLE`, `INVENTORY`, `PORTAL`, `CONVEYOR`, `CUSTOM`), not a constant. The single example makes it look fixed. Worth stating explicitly, since a consumer switching on `type` needs to expect all five.

data fields are conditional on tagMetaData

The example shows all 14 fields at once, which is misleading - most only appear when requested:

Configuration	data fields present
No tagMetaData	eventNum, format, idHex - 3 only
["ANTENNA","RSSI","CHANNEL","PHASE","SEEN_COUNT","PC","CRC","XPC"]	11
["ANTENNA","RSSI","TID","USER","XPC","PC","CRC"]	9, incl. TID and USER

Nothing in the document indicates that a plain inventory yields only three fields. An integrator reading the example would expect `antenna` and `peakRssi` to always be present.

TID and USER confirmed; XPC never appeared

All three are accepted as `tagMetaData` values (HTTP 200 each). `TID` and `USER` then populate:

```
{"TID": "e2806894200040017790e471",
 "USER": "Error: tag returned error code 0x03 = Memory overrun"}
```

`TID` returns real data. The `USER` error is a property of the test tag, which has no user memory - not an API fault, and useful to show, since the field carries an error string rather than being omitted.

`XPC` was requested in two separate configurations and never appeared in any of the 228 captured events. Either this tag population has no XPC data, or the field is not emitted. Undetermined - but the documented example shows `"XPC": "string"`, which is a placeholder rather than a real value, so the field's real shape is unverified in both the document and here.

The other two tag data paths were not exercised

Path	Status
/tagDataEvents	[OK] verified - envelope and 11 fields
/mode_tag_data_events	[-] shows the bare data object with no envelope; not reproduced
/directionality_tag_data_ev	[X] not supported on FXR60 or FXR90 - see section 5

`/mode_tag_data_events` documents the same fields **without** the `type`/`timestamp` wrapper. Every event captured here had the wrapper, so it is unclear when the unwrapped form applies - that distinction is not explained in either description.

`/directionality_tag_data_events` **cannot be produced on either model** - see section 5.

---

5. [BUG] Directionality events are documented for hardware that does not support them

`/directionality_tag_data_events` states:

Property	Value
Applies To	FXR60 / FXR90

Both readers disagree. From `GET /cloud/readerCapabilities`:

Reader	directionalitySupported	tagLocationingSupported
10.233.48.36 - FXR60, app 5.0.7	false	false
10.233.48.49 - FXR90, app 5.0.4	false	true

And the mode is rejected outright on both:

```
PUT /cloud/mode  {"type":"DIRECTIONALITY","antennas":[1],"transmitPower":30}

FXR60  -> 422  "Directionality operating mode is Not Supported"
FXR90  -> 422  "Directionality operating mode is Not Supported"
```

The firmware is unambiguous and consistent - a dedicated capability flag, plus a clear rejection naming the mode. The documentation is what is wrong.

Consequences. The event page, and the `zoneHistory` / `locationHistory` pages it links to, describe a payload no FXR60 or FXR90 can emit. An integrator following it will design zone-transition logic, then discover at `PUT /cloud/mode` that the mode does not exist on the platform. The `tagDataEvents` schema also lists `DIRECTIONALITY` among its `type` values, so the impression is reinforced in three places.

One nuance worth keeping. `tagLocationingSupported` is **`true` on the FXR90** and `false` on the FXR60. So tag locationing is not uniformly unavailable - only *directionality* is unsupported on both. `locationHistory` may therefore be reachable on an FXR90 through some other path; that was not tested here and the flag alone does not prove it.

Ask. Either correct "Applies To" to name the models that do support directionality, or mark the page as not applicable to FXR60/FXR90. If the mode is planned but unimplemented, say so. And since `tagLocationingSupported` differs between the two models, capability differences should be stated per model rather than as a single combined "FXR60 / FXR90".

Summary

Finding	Impact
6 of 7 event types have no example	A consumer cannot know the data shape for gpi, gpo, error, warning, firmwareUpdateProgress, userapp
Heartbeat gains ~9 fields not documented	Including interfaceConnectionStatus and dataPathStatistics - the most useful monitoring content
Heartbeat loses 4 documented counters	Moved into dataPathStatistics[]; clients reading the old paths get nothing
systemTime vs systemtime	Casing mismatch; a consumer keying on the documented name gets nothing
flash documented as populated	Actually null
Two time formats in one event	timestamp is ISO-8601, systemtime is DD/MM/YYYY HH:MM
Tag data envelope + 11 fields	[OK] accurate - verified live
Tag data type looks fixed	Shows SIMPLE; actually reflects the mode (CUSTOM observed)
Tag data example shows all 14 fields	Most are conditional on tagMetaData; a plain inventory returns 3
XPC unverified	Requested twice, never emitted; the example value is the placeholder "string"
Directionality documented for unsupporting hardware	[BUG] directionalitySupported: false on both models; PUT /cloud/mode returns 422 "Directionality operating mode is Not Supported" - yet the page says "Applies To: FXR60 / FXR90"

What would fix it

1. **Add an example per event type.** Seven small examples, one per `type` value. The `gpo` payload above is three fields.
2. **Regenerate the heartbeat example from a current reader** rather than editing the existing one - the structural changes (`dataPathStatistics`, `interfaceConnectionStatus`) are easy to miss by hand.
3. **Fix `systemTime` -> `systemtime`**, or change the firmware to match the document. One of the two.
4. **Confirm the status of `flash`** - populated on some hardware, or deprecated.
5. **State that `antennas` and `numTagReadsPerAntenna` are device-dependent**, so the 13-entry example is not read as fixed.
6. **Tag data:** note that `type` carries the operating mode, and mark which `data` fields require a `tagMetaData` entry. The current example implies all 14 are always present when a plain inventory returns three.
7. **Explain when `/mode_tag_data_events` (unwrapped) applies** versus `/tagDataEvents` (wrapped in `type`/`timestamp`). Every event observed here was wrapped.
8. **Replace the `"string"` placeholders** in the directionality and `XPC` examples with real captured values.
9. **Correct the directionality "Applies To" field.** Neither FXR60 nor FXR90 supports the mode. State capabilities per model - `tagLocationingSupported` is `true` on FXR90 and `false` on FXR60, so a combined "FXR60 / FXR90" label is misleading either way.

Caveats

- Captured from **one reader**, FXR60 app 5.0.7. Field sets may differ on FXR90 or other firmware - particularly `flash` , `powerSource` and the antenna count.
- Directionality was tested by capability flag and by mode rejection on both models. It was **not** tested on any other Zebra reader, so which platforms do support it is unknown from here.
- Tag data was captured on a small tag population through **antenna 1 only**, so `XPC` absence and the `USER` memory error may not generalise. The directionality and unwrapped-mode paths were **not** exercised at all.
- Only `heartbeat` and `gpo` were observed. `gpi` needs a physical input change; `error` and `warning` need a fault condition; `firmwareUpdateProgress` needs an OS update; `userapp` needs an installed application emitting events. **Those five shapes remain unverified here** - this report does not claim to document them, only that the file does not either.
- The reader was restored to its original `endpointConfig` after capture; `DATA_AWS_CLOUD_EVENTS` reconnected and verified.